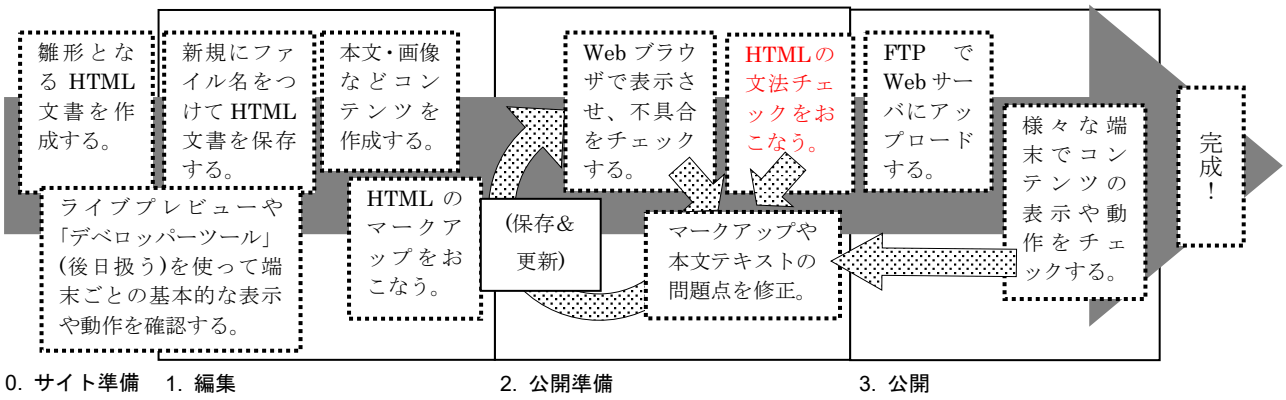
第３回：HTML 文書作成の基礎練習 (2)

担当：千葉庄寿（schiba@reitaku-u.ac.jp）

本日の内容

- HTML の構造と文法
- HTML の主な要素と属性(特に画像、リンク関係)
- HTML 特有の表現

参考１：HTML 文書および Web コンテンツの作成の一般的な手順



※ 0. の雛型ファイルの作成とサイト管理については後日実習します。

参考 2：学生用 Web サーバのサイト設定と公開 URL

学生用 Web サーバのサイト設定

- **Web サーバ (学生公開用)**：www.cs.reitaku-u.ac.jp
- **アップロード先ディレクトリ**：/home/User-ID/www/
  - User-ID は各自のユーザ ID
  - www フォルダは半角小文字で作成する
- 学生から Web サーバに FTP するためには VPN 接続をおこなう必要があります。

重要：**Web サーバの公開 URL の参照**：

- 学内ネットワークから課題ページを参照するための URL (以下は helsinki.html の場合)：
  https://www.cs.reitaku-u.ac.jp/~User-ID/helsinki.html
  - User-ID は各自のユーザ ID
- **学外ネットワーク**(携帯端末の回線や自宅)**から参照するための URL** は後日お知らせします。現時点では、公開したコンテンツは olive を含む学内ネットワークからのみ参照できます。(学外から参照するためには VPN 接続をおこなう必要があります。)

★ myfirstpage.html と helsinki.html を公開しましょう

第 2 回授業資料を参照して myfirsftpage.html, helsinki.html の 2 つの Web ページを編集し、Web サーバに公開してください。更新した内容は、再度 FTP をおこなって、公開されているリソースの更新をおこなう必要があることに注意してください。

2 つのファイルは 10 月 15 日(木)午後 7 時までに公開をすませてください。FTP ツールの導入、設定に問題がある人は、それまでに必ず解決すること。

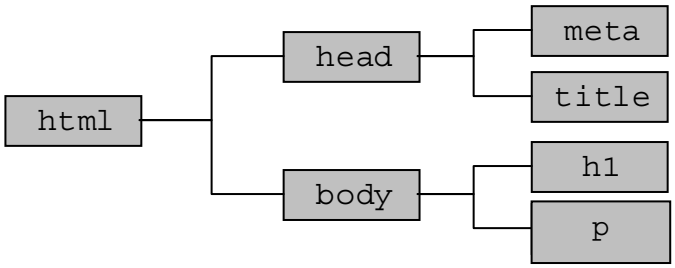
公開 URL のリストは Moodle の第 2 回授業トピックにあります。

- 学内ネットワーク(olive を含む)から参照するための課題ページの URL：
  http://www.cs.reitaku-u.ac.jp/~User-ID/myfirstpage.html
  http://www.cs.reitaku-u.ac.jp/~User-ID/helsinki.html
  - User-ID は各自のユーザ ID

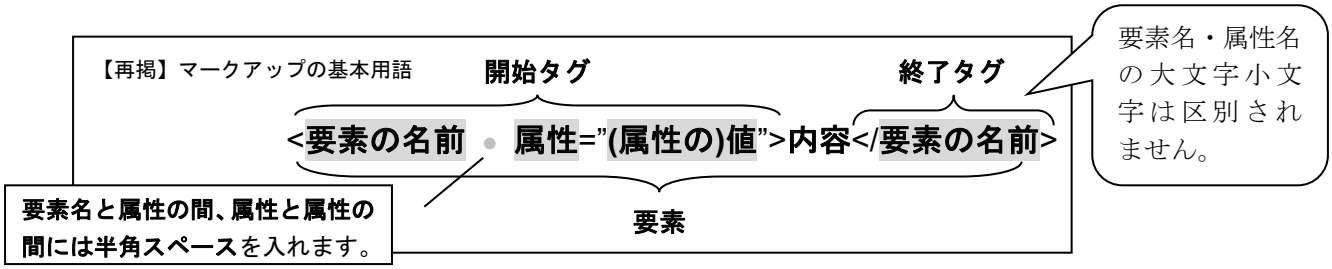
1. HTML の構造と文法

1.1. HTML のツリー構造(復習)

HTML 文書は、ある**要素（開始タグ** <...> start tag で始まり**終了タグ**</...> end tag で終わるまでの部分) の中に他の要素を「**入れ子**」にしていく形で作成される。これを使い、HTML 文書は html 要素を出発点として、親要素から子要素に枝分かれしていく階層構造、つまり「**ツリー構造**」として表現することができる(**右図**)。HTML 文書の内容が複雑になると、要素の入れ子関係も当然複雑になる。



HTML のツリー構造の例



※ 属性は、1 つのタグに複数入れることができる。要素と属性の間と同様、属性と属性の間も半角スペース(またはタブ、改行)で区切ること。

1.2. HTML の文法

HTML には、大きく分けて以下のような種類の文法エラーがある。

1. **文字種の間違い**：タグの文字を間違って全角で入力した(なお、**タグの大文字・小文字の区別はない**が、以前のバージョンの HTML には小文字のみをタグに使用するものがあった)。
2. **HTML の要素名・属性名の選択間違い**：使用している HTML のバージョンが非推奨としている古い要素や属性を使用してしまった。
3. **HTML のタグ規則違反**：終了タグの書き忘れ、閉じ括弧(>)の書き忘れ、引用符の閉じ忘れ、要素や属性間のスペースの書き忘れ、終了タグのスラッシュ / の書き忘れ等。
   ★ HTML にはタグを省略してよい要素がある。省略可能なタグは厳密に決められており、以下の Web ページにリストされている：
   `https://html.spec.whatwg.org/multipage/syntax.html#optional-tags`
   ★ img や meta、br など空要素は、タグ末尾にスラッシュ / を入れてもよい(<img …>, <meta… />,
)。これは、古い HTML の文法に従ったもので、文法チェッカーによっては空要素に / が入っていると警告を出すことがある。
4. **タグのスペルミスや文字種の間違い**：<blockquote> を <brockquote>、<link> を <rink>、と打ってしまった。<address> を<adress> と打った。半角スペースと間違えて全角スペースを入力した、など。
5. **要素の入れ子関係の間違い**：title 要素を body 要素の中に書いてしまった、strong 要素の中に p 要素を入れてしまった、など。
   × 私はこれを強く主張したい！
   ○ 私はこれを強く主張したい！
   × <h1>千葉のページにようこそ！</h1>

★インラインレベルとブロックレベルの要素の違いについては§2.1 を参照。なお、現在の HTML では、a 要素はブロックレベルの外にも記述できるようになっている(以前のバージョンでは a 要素はインラインレベルだった)。

以下は最新の HTML LIVING STANDARD では OK (古いバージョンでは×)

<a●href="…"><h1>サイトタイトル</h1> サイトへようこそ！

上記のようなリンクを実現させるためには、古い HTML では、以下のような書き方をする必要があった。

<h1><a●href="…">サイトタイトル</h1>

<a●href="…">サイトへようこそ！

6. **タグの交差**：子要素を閉じる前に親要素を閉じてしまった。
   × 私はこれを強く主張したい！ 。
   ○ 私はこれを強く主張したい！ 。
7. **順序や回数の間違い**：1 回しか使えない head, body 要素を何度も書いてしまった、id 属性の値として同じものを複数箇所に書いてしまった。

タグを人間が入力する以上、上記のような間違いは常に起こりうる。全ての文法エラーを目で発見し修正するのは効率が悪い。

作成したコードが HTML や CSS などの規則に従っているか確認する作業のことを**「検証」(バリデート validate)**という。コードを読んでもエラー箇所が分からない場合や、HTML や CSS のエラーをまとめて確認・修正したい場合、バリデーションサービス(以下**文法チェックサービス**と言う)を利用すると便利である。(コードの編集途中に出てくる lint のメッセージは、バリデートによく似ているが、厳密な文法チェックではない。)

検証結果には、文法エラーのほか、警告も含まれる(Another HTML-lint 5 では、警告と文法エラーは減点の大きさで表現される)。警告は、必ずしも不正な HTML であることを意味するわけではないが、指示を理解しコードを改善することが期待される。

授業では、以下の VS Code の拡張機能と HTML の文法チェックサイトを用いる。日本語でチェックしたい人は「Another HTML-lint 5 ゲートウェイ」を使用するとよい。これらのサイトは Chrome の「ブックマーク」に入れておき、すぐに呼び出せるようにしておくとよい：

- HTMLHint：
  ★HTML のエラー箇所と内容を表示してくれる VS Code 拡張機能。表示は英語。自動修正機能もあるが、ツールに頼り切りにはらず、まずはエラーを確認するために使用してみよう。
  `https://marketplace.visualstudio.com/items?itemName=HTMLHint.vscode-htmlhint`
  VS Code で拡張機能「HTMLHint」(後述)を導入すると編集中に現れる黄色い波線は、その行(ないしその前)のコードにエラーがあることを表わす。破線のあるコードにカーソルをあてるか、「問題」パネルを開くことでエラーの内容が表示される。なお、HTMLHint には、軽微な警告は表示されない(以下の Nu Html Checker, Another HTML-lint 5 は細かな警告を出してくれる)。
- Nu Html Checker：
  ★HTML の仕様策定に長年関わっている World Wide Web Consortium (W3C)が提供する文法チェッカー。学内むけの URL を直接読み込ませることはできないので、「file upload」または「text input」を用いる。言語は英語のみ。「check as CSS」をチェックすると CSS もチェックできる。
  `https://validator.w3.org/nu/`
- Another HTML-lint 5 ゲートウェイ：
  `https://www.htmllint.net/html-lint/htmllint.html`
  ★学内むけの URL を直接読み込ませることはできない。「FILE」または「DATA」を用いること。W3C の公式サービスではないが、日本語が使える。

これらのオンラインサイトを使って自分で作成した Web ページの文法チェックをおこなうためには、保存している HTML 文書をアップロードする(学生用 Web サーバは大学外のネットワークから参照できないので、URL では文法チェックができないことに注意)。

なお、W3C は CSS の文法チェックサービス(CSS Validation Service) [1] も提供している。CSS のバリデーターは多言語対応しており、日本語インターフェースがある。

2. 本日の実習・課題

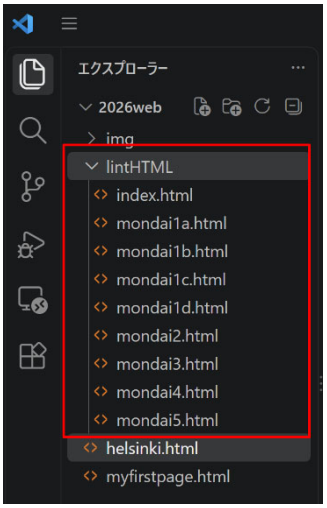
課題 1：本日の資料をよく読み、バリデートと HTML の修正の練習をする

1. VS Code を起動し、前回授業で設定したフォルダ「2026web」を読み込む。制限モードで開いていると拡張機能が使えないので、フォルダーを「信頼済み」にすること。



2. Moodle コースから実習ファイル「lintHTML.zip」をダウンロードし、展開する。展開した「lintHTML」フォルダを［2026web］フォルダに**フォルダごと**コピーする。

3. VS Code の「エクスプローラ」の表示を更新し、「lintHTML」フォルダが表示されていることを確認する(右図)。



4. 「lintHTML」フォルダに入っている index.html ファイルを VS Code で開く。index.html の指示(コードの中で指示が読みにくい時は Live Preview か Web ブラウザでないようを表示させるとよい)に従って「lintHTML」フォルダ内の各ファイルを開き、HTMLHint や文法チェックサービスを使ってエラー内容を確認、修正するとともに index.html の記録欄にエラーの内容をメモしなさい(エラーの種類の分類については、§1.6 を参照すること)。

5. 編集をおこなった index.html を保存し、Moodle の課題提出場所に提出しなさい。最低限、mondai1a～1d (およびそれらがまとまっている mondai2.html の修正にも挑戦するとよい)までは必ずチェックすること。

課題 2：これまでに作成した HTML 文書のバリデートをおこなう

6. `myfirstpage.html` および `helsinki.html` の文法が正しいかどうか、文法チェックサイトを使って確認してみよう。Web ブラウザで表示できれば HTML の文法は正しいとは限らないので注意。

7. チェック結果をよく読み、必要ならばマークアップを修正しよう。(修正した場合には、ページ下のフッターにある公開日付を更新しておくとよい。)

8. 修正した HTML 文書を FTP でアップロードし、公開しているページを更新しよう。

9. HTMLHint、Nu Html Checker、「Another HTML-lint 5 ゲートウェイ」のチェックがどのように異なるか、比較してみよう。あえて間違ったコードをつくって結果を検証してみるとよい。

[1] `http://jigsaw.w3.org/css-validator/`

課題 3：viewport の効果を確認する

10. ビューポートの効果を確認するためのサンプルページをモバイルサイトで閲覧してみよう。

- ビューポートなし 
- ビューポートあり 

ビューポート viewport は仮想的なウィンドウサイズのことで、この幅が実際の画面よりも大きければ、コンテンツは画面からはみ出して表示され一部しか表示されない状態になる。逆に、ビューポートが小さければ、画面には余白ができる。

「ビューポートあり」のページには、meta 要素で以下のようにビューポートが記述されている。

```
<meta name="viewport" content="width=device-width, initial-scale=1">
```

width は、ビューボートが最小でどのような大きさかを設定する。**device-width は、端末の画面の物理的なサイズ**(実際の画面の解像度ではないことが重要！)**を表わす**。この設定により、ビューポートの幅が、端末やブラウザ・アプリの物理的な幅に応じて柔軟に設定される。initial-scale はページを読み込んだ時の表示倍率で、標準は 1 (拡大・縮小なし)。この 2 つの設定により、一般的なモバイル端末で Web ページを適切に表示することができる。

課題 4：myfirstpage.html の修正：table の作成

11. 実習課題 myfirstpage.html を以下のように編集しなさい。適宜上書き保存すること。
    - 「こんにちは。」で始まるウェルカムメッセージと、第 2 回で追加したおすすめ YouTube 動画の情報の後に、以下のように新たな見出しと table 要素を新たに挿入し、表を使ってオススメ映画(またはその他のトピックでもよい)の情報を表示させなさい。

```
<h2>おすすめ情報</h2>
<caption>私のおすすめ映画ベスト 3</caption>
<table border="1">
<tr><th>タイトル</th><th>公開年</th><th>おすすめポイント</th></tr>
<tr><td>映画のタイトル</td><td>公開年をここに</td><td>オススメ理由など</td></tr>
<tr><td>映画のタイトル</td><td>公開年をここに</td><td>オススメ理由など</td></tr>
<tr><td>映画のタイトル</td><td>公開年をここに</td><td>オススメ理由など</td></tr>
<tr><td colspan="3" style="text-align: right">最終更新：2026-10-09</td><tr>
</table>
```

    - 上記は「おすすめ映画」となっているが、テーマは好きなものに変えてよい。
    - タイトルないしおすすめ理由のテキスト中に適宜リンクを作成して、公式サイトなど必要な情報にアクセスできるようにするとよい(加点します)。
    - 表は、border="1" という属性 [2] を記述して、1px の枠線を引くように設定している。この指定だけだと枠が 2 重線で表現されるように見える。これは、**表全体の枠線 border と、セルの**

[2] この属性は、本来であれば CSS でスタイルとして記述すべきものであり、最新の HTML Living Standard では廃止されている。そのため、文法チェックをおこなうと警告が出るが、今回の実習では無視してよい。

枠線の両方が 1px で引かれるためである。この線を 1 本にまとめるためには、以下のように、スタイルシートを記述する style 属性を使ってスタイルシート(CSS)のプロパティ border-collapse: collapse を適用する。border 属性と style 属性はスペースで区切ることに注意。
<table●border="1"●style="**border-collapse: collapse**">

- 表の最後の行では、colspan 属性を使用して、横にならんだセル 3 つを 1 つにまとめている(縦方向にまとめるには rowspan 属性を使用する)。また、style 属性で text-align プロパティを使い、テキストを右揃えに配置している。colspan 属性と style 属性はスペースで区切ることに注意。

12. 作成した myfirstpage.html を FTP で Web サーバに再度アップロードし、公開している内容を更新しなさい。アップロードしたら、授業ホームページにある課題のリンクからコンテンツが正しく閲覧できることを確認しよう。

課題 5：helsinki.html の修正：img タグに a タグを追加

13. 第 2 回で作成した実習課題 helsinki.html を以下のように編集しなさい。適宜上書き保存すること。
    - img 要素で表示した画像 church_w400.jpg をクリックすると大きな画像 church_w1600.jpg (が表示されるようにリンクタグを追加しなさい。

```
<p><a●href=”church_w1600.jpg”●title=”大きな画像を表示します”><img●src=”church_w400.jpg”●alt=”ヘルシンキ大聖堂”></a></p>
```

    img 要素を a 要素で囲むようにマークアップすることに注意。img 要素を選択後、VS Code の拡張機能「htmltagwrap」の機能を使い、ショートカット Alt + W (macOS は Option + W) でタグを Wrap 挿入すると簡単である。
    - 上記コードを参考に、a 要素に title 属性を記述すること。
    - Web ブラウザでプレビューし、リンクが機能することを確認しなさい。また、カーソルを画像のうえに置いた時に「大きな画像を表示します」という title 属性の値が表示されることを確認しなさい(右図)。
    - 編集した helsinki.html を文法チェックし、エラーがあれば修正をおこない、正しい文法の HTML 文書として仕上げなさい。



14. 作成した helsinki.html を FTP で Web サーバに再度アップロードし、公開している内容を更新しなさい。Web ページに使用している画像ファイルもアップロードする必要があるので注意。アップロードしたら、授業ホームページにある課題のリンクからコンテンツが正しく閲覧できることを確認しよう。

(以上)